Chapter 11: Getting Started with Pygame with Pong

Contents

Chapter 11: Getting Started with Pygame with Pong 1
DRY 2
Code Shape 2
  Visualize and Debug Programs 2
What is Pygame? 2
Install and Update Pygame with pip 2
Tutorial 1: Hello World 3
Pong 5
Pong 1: Object Oriented Pong 5
Pygame Core Concepts 7
  Game Loops and Game States 8
  Quitting the Game Loop 10
  Event Objects 11
  Pixel Coordinates 12
  Colors 13
  Drawing Functions 15
  Frames per Second 16
Pong 2: Stationary Ball 17
  Rect Object 20
  Primitive Drawing Functions 22
  Animation 22
Pong 3: Moving Ball 24
Pong 4: Moving Ball Class 26
Pong 5: Collisions 30
  Assignment Submission 30

Time required: 90 minutes

DRY

Don't **R**epeat **Y**ourself

Code Shape

Please group program code as follows.

- Declare constants and variables
- Get input
- Calculate
- Display

Visualize and Debug Programs

The website www.pythontutor.com helps you create visualizations for the code in all the listings and step through those programs one instruction at a time. As you do that, pay attention to the movements of the red and green arrows on the left, the diagram on the right, and the printed material at the bottom. This will help you to better understand the behavior of the programs we are working on.

This is a great way to debug your code. You can see the variables change as you step through the program.

www.pythontutor.com

What is Pygame?

The Pygame library is the most well-known python library for making games. It's not the most advanced or high-level library. It is simple and easy to learn (comparatively). Pygame serves as a great entry point into the world of graphics and game development.

Pygame is installed with Python. Pygame is a framework that includes several modules with functions for drawing graphics, playing sounds, handling mouse input, and other game like things. Pygame provides functions for creating programs with a **graphical user interface,** or **GUI** (pronounced, "gooey").

Install and Update Pygame with pip

pip is a package manager for **Python**. It's a tool that allows you to install and manage additional libraries and dependencies that are not distributed as part of the standard library.

Let's start by updating pip. Open a command prompt, type in the following command:

```
pip install --upgrade pip
```

Install Pygame:

```
pip install pygame
```

If Pygame is already installed, you may want to update Pygame:

```
pip install pygame -U
```

You are now ready to embark on your successful and prosperous game development career in Python!

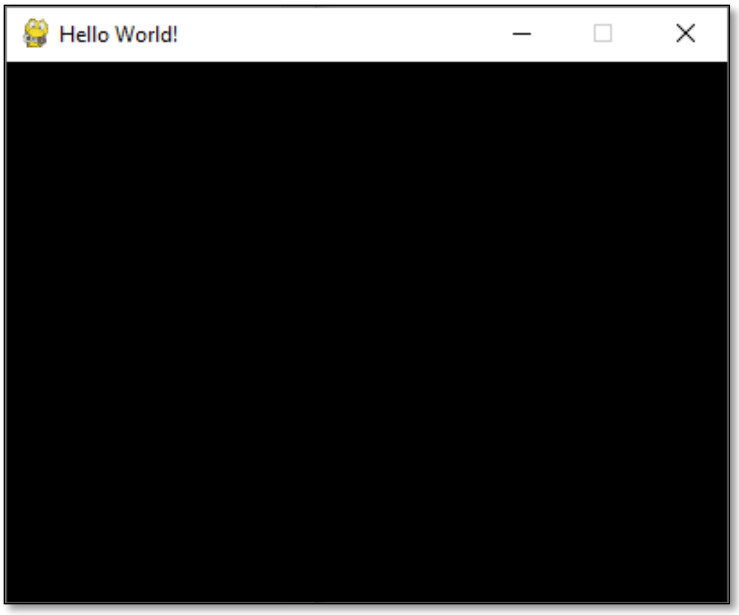
Tutorial 1: Hello World

Yet another Hello, World program. This time we do it in Pygame.

Type in the following code and save the file as **hello_world.py**.

```python
"""
    Name: hello_pygame.py
    Author:
    Date:
    Purpose: Create a blank Pygame window
"""
# Import pygame libraries
import pygame
# Import sys for sys.exit() to cleanly exit program
import sys

#--------------- INITIALIZE PYGAME -----------------------#
# Initialize the pygame module
pygame.init()
# Create display surface
SURFACE = pygame.display.set_mode((400, 300))
# Set caption for window
pygame.display.set_caption("Hello Pygame World!")

#--------------- INFINITE GAME LOOP ----------------------#
while True:
    # Listen for program events
    for event in pygame.event.get():

        # Closing the program window
        # causes the QUIT event to be fired
        if event.type == pygame.QUIT:
            # Exit Pygame
            pygame.quit()
            # Exit the program
            sys.exit()

    # Redraw the screen
    pygame.display.update()
```

Example run:



Yay! You've just made the world's most boring video game! It's just a blank window with Hello World! at the top of the window in the title bar.

Creating a window is the first step to making graphical games.

Pong

Imagine the year is 1973. November 29th. All the talk is about a new arcade game which has been released by Atari. That game is called Pong.

It might not seem much of a game by today's standards. It was a massive hit in its day.

Don't be deceived, although a simple game, Pong covers a wide range of aspects of computer game programming. There is movement, control, collision detection, scoring, artificial intelligence. It's all in there!

Being able to program Pong is a doorway to being able to program a lot of other games.

Watch out! Once you start playing Pong you might find less time to program, as it is quite addictive!

Pong 1: Object Oriented Pong

To program Pong without getting into a big mess, we are going to refactor the previous program with classes and objects. Don't worry if it doesn't make sense now. We will cover this material as we go. Object Oriented Programming allows us to create more easily maintainable complex programs.

```python
"""
    Name: pong1.py
    Author:
    Date:
    Purpose: Object Oriented Pong
    Extablish the playing field
"""

# Import pygame libraries
import pygame
from pygame.locals import *
# Import sys.exit to exit program
from sys import exit


class Pong:
    def __init__(self):
        """ Initialize the Pong class """
        # Color Constant
        self.COUGAR_GOLD = (249, 190, 0)

        # Initialize Pygame for action
        pygame.init()

        # Create display surface with title bar caption
        self.SCREEN = pygame.display.set_mode((1024, 768))
        pygame.display.set_caption("Pong")

```

```
#---------------------------- RUN GAME ----------------------------#
    def run_game(self):
        """
            Infinite Game Loop
        """
        while True:
            # Listen for program events
            for event in pygame.event.get():

                # Closing the program by clicking the X
                # causes the QUIT event to be fired
                if event.type == QUIT:
                    # Quit Pygame
                    pygame.quit()
                    # Exit Python
                    exit()

            # Fill the display surface with Cougar gold
            self.SCREEN.fill(self.COUGAR_GOLD)

            # Redraw the surface object
            pygame.display.update()


#------------------------- MAIN PROGRAM ----------------------------#
# Create game instance/object
pong = Pong()
# Start the game with the run_game method
pong.run_game()
```

Pygame Core Concepts

Let's start by explaining several core concepts related to the Pygame library and about creating games in general. Many of these concepts are transferable skills. If you switch to a more advanced game engine, many of these concepts will still hold true.

```
import pygame, sys
from pygame.locals import *
```

The above code imports **pygame** and its modules into the program. The second line allows us to use the functions and variables in the **pygame.locals** module without having to add the lengthy **pygame.locals** prefix. The **sys** module is imported to exit Pygame.

Normally if you want to call a function that is in a module, you must use the **modulename.functionname()** format after importing the module. However, with **from modulename import ***, you can skip the **modulename.** portion and simply use **functionname()** (just like Python's built-in functions).

The reason we use this form of import statement for **pygame.locals** is **pygame.locals** contains several constant variables that are easy to identify as being in the **pygame.locals** module without **pygame.locals.** in front of them.

```
pygame.init()
```

This line is necessary whenever you want to use the Pygame library. Think of it as starting the pygame engine. It must be added before any other **pygame** function, or else an initialization error may occur.

```
# Create a display surface referenced by the surface reference variable
screen = pygame.display.set_mode((1024, 768))
```

This is a call to the **pygame.display.set_mode()** function, which returns the **pygame.Surface** object for the window. (Surface objects are described later.) Notice that we pass a tuple value of two integers to the function: **(1024, 768)**. This tuple tells the **set_mode()** function how wide (x) and how high (y) to make the window in pixels. **(1024, 768)** will make a window with a width of 1024 pixels and height of 768 pixels.

An important aspect in games is individually accessing coordinates. To show a set of co-ordinates, you place both the X and Y values in a tuple, where the first integer is X and second integer is Y.

Both values start from the top-left hand side. X values increase from left to right, and Y values increase from top to bottom.

The coordinate values must be integers. A pixel is meant to represent the smallest possible area on a screen. There is no such thing as “half a pixel”.

The **pygame.Surface** object (we will just call them Surface objects for short) returned is stored in a reference variable named **screen**.

```
# Sets the title bar caption
pygame.display.set_caption('Pong')
```

This sets the caption text that will appear at the top of the window by calling the **pygame.display.set_caption()** function. The string value 'Pong' is passed in this function call to make that text appear as the caption of the window.

Game Loops and Game States

Every game that has what is called a “Game Loop”. It’s a standard game development concept.

The Game Loop is where all the game events occur, update, and get drawn to the screen. Once the initial setup and initialization of variables is out of the way, the Game Loop begins where the program keeps looping over and over until an event of type **QUIT** occurs.

Changes in the game are not implemented until the **pygame.display.update()** has been called. Since games are constantly changing values, the update function is in the game loop, constantly updating.

```
#Game loop runs forever
while True:
    # Code
    # Code

    pygame.display.update()
```

This is a while loop that has a condition of simply the value **True**. This means that it never exits due to its condition evaluating to False. The only way the program execution will ever exit the loop is if a break statement is executed (which moves execution to the first line after the loop) or **sys.exit()** (which terminates the program). If a loop like this was inside a function, a return statement will also move execution out of the loop (as well as the function too).

A game loop (also called a main loop) is a loop where the code does three things:

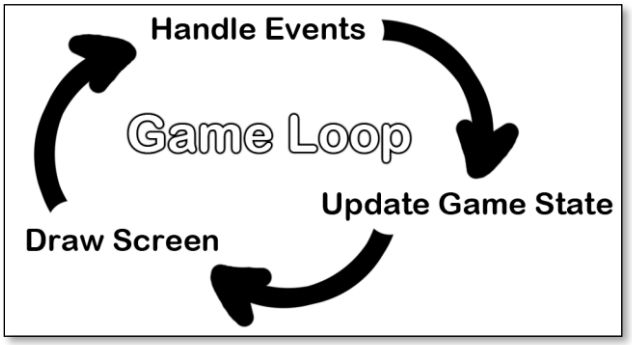
1. Handles events
2. Updates the game state
3. Draws the game state to the screen

The game state is simply a way of referring to a set of values for all the variables in a game program. In many games, the game state includes the values in the variables that tracks the player's health and position, the health and position of any enemies, which marks have been made on a board, the score, or whose turn it is. Whenever something happens like the player taking damage (which lowers their health value), or an enemy moves somewhere, or something happens in the game world we say that the game state has changed.

If you've ever played a game that let you save, the "save state" is the game state at the point that you've saved it. In most games, pausing the game will prevent the game state from changing.

Since the game state is usually updated in response to events (such as mouse clicks or keyboard presses) or the passage of time, the game loop is constantly checking and re-checking many times a second for any new events that have happened. Inside the main

loop is code that looks at which events have been created (with Pygame, this is done by calling the **pygame.event.get()** function). The main loop also has code that updates the game state based on which events have been created. This is usually called event handling.



Quitting the Game Loop

Every game loop must have an end point, or some action that triggers the end point (such as clicking the quit button), or else your game will run indefinitely.

```
while True:
    pygame.display.update()
    for event in pygame.event.get():
        if event.type == QUIT:
            pygame.quit()
            sys.exit()
```

We call both **pygame.quit()** and **sys.exit()** to close the pygame window and the python script respectively.

Event objects have a member variable (also called attributes or properties) named **type** which tells us what kind of event the object represents. Pygame has a constant variable for each of possible types in the **pygame.locals** modules. Line 9 checks if the Event object's type is equal to the constant QUIT. Remember that since we used the **from pygame.locals import *** form of the import statement, we only have to type **QUIT** instead of **pygame.locals.QUIT**.

If the Event object is a quit event, then the **pygame.quit()** and **sys.exit()** functions are called. The **pygame.quit()** function is sort of the opposite of the **pygame.init()** function: it runs code that deactivates the Pygame library. Your programs should always call **pygame.quit()** before they call **sys.exit()** to terminate the program..

Since we have no if statements that run code for other types of Event objects, there is no event-handling code for when the user clicks the mouse, presses keyboard keys, or causes

any other type of Event objects to be created. The user can do things to create these Event objects, but it doesn't change anything in the program because the program does not have any event-handling code for these types of Event objects. After the for loop on is done handling all the Event objects that have been returned by **pygame.event.get()**, the program execution continues to these final lines. We will add more event handling as we grow our program.

```
# Redraw the surface object
pygame.display.update()
```

This line calls the **pygame.display.update()** function, which draws the Surface object returned by **pygame.display.set_mode()** to the screen (remember we stored this object in the **window** variable). Since the Surface object hasn't changed (for example, by some of the drawing functions that are explained later in this chapter), the same image is redrawn to the screen each time **pygame.display.update()** is called.

That is the entire program. After this line is done, the infinite while loop starts again from the beginning. This program does nothing besides make a window appear on the screen, constantly check for a **QUIT** event, and then redraws the unchanged window to the screen over and over and over again.

Event Objects

Any time the user does one of several actions such as pressing a keyboard key or moving the mouse on the program's window, a `pygame.event.Event` object is created by the Pygame library to record this "event". (This is a type of object called Event that exists in the event module, which itself is in the pygame module.) We can find out which events have happened by calling the **pygame.event.get()** function, which returns a list of **pygame.event.Event** objects (which we will just call Event objects for short).

The list of Event objects will be for each event that has happened since the last time the **pygame.event.get()** function was called.

```
# Watch for any program events
for event in pygame.event.get():

    # Closing the program by clicking the X
    # causes the QUIT event to be fired
    if event.type == QUIT:

        # Quit Pygame
        pygame.quit()

        # Stop Python script
        sys.exit()
```

This is a for loop that will iterate over the list of Event objects that was returned by **pygame.event.get()**. On each iteration through the for loop, a variable named event will be assigned the value of the next event object in this list. The list of Event objects returned from **pygame.event.get()** will be in the order that the events happened. If the user clicked the mouse and then pressed a keyboard key, the Event object for the mouse click would be the first item in the list and the Event object for the keyboard press would be second. If no events have happened, then **pygame.event.get()** will return a blank list.

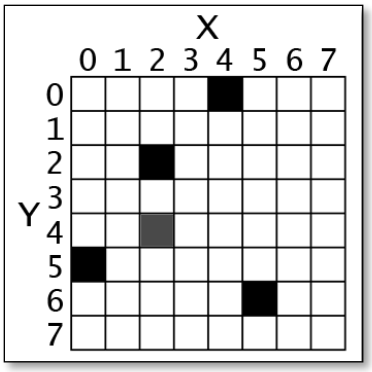
An "Event" occurs when the user performs a specific action, such as clicking their mouse or pressing a keyboard button. Pygame records every event that occurs.

One of the many attributes (or properties) held by event objects is type. The type attribute tells us what kind of event the object represents.

If you take a look at the example shown earlier, you'll see we used **event.type == QUIT** to determine whether the game is to be closed or not.

Pixel Coordinates

The window that the "POng" program creates is composed of little square dots on the screen called pixels. Each pixel starts off as black but can be set to a different color. Imagine that instead of a Surface object that is 300 pixels wide and 300 pixels tall, we just had a Surface object that was 8 pixels by 8 pixels. If that tiny 8x8 Surface was enlarged so that each pixel looks like a square in a grid, and we added numbers for the X and Y axis, then a good representation of it could look something like this:



We can refer to a specific pixel by using a Cartesian Coordinate system. Each column of the X-axis and each row of the Y-axis will have an “address” that is an integer from 0 to 7 so that we can locate any pixel by specifying the X and Y axis integers.

For example, in the above 8x8 image, we can see that the pixels at the XY coordinates (4, 0), (2, 2), (0, 5), and (5, 6) have been painted black, the pixel at (2, 4) has been painted gray, while all the other pixels are painted white. XY coordinates are also called points. If you’ve taken a math class and learned about Cartesian Coordinates, you might notice that the Y-axis starts at 0 at the top and then increases going down, rather than increasing as it goes up. This is just how Cartesian Coordinates work in Pygame (and almost every programming language).

The Pygame framework often represents Cartesian Coordinates as a tuple of two integers, such as (4, 0) or (2, 2). The first integer is the X coordinate and the second is the Y coordinate.

Colors

Colors are a big part of any game development framework or engine.

Pygame uses the RGB system of colors. This stand for Red, Green and Blue respectively. These three colors combined (in varying ratios) are used to create all the colors you see on computers, or any device that has a screen.

Some of these colors are predefined in Pygame. See the **pygame_color_picker.py** program attached to this assignment.

There are three primary colors of light: red, green and blue. (Red, blue, and yellow are the primary colors for paints and pigments, but the computer monitor uses light, not paint.) By combining different amounts of these three colors you can form any other color.

In Pygame, we represent colors with tuples of three integers. The first value in the tuple is how much red is in the color. An integer value of 0 means there is no red in this color, and a value of 255 means there is the maximum amount of red in the color. The second value is for green and the third value is for blue. These tuples of three integers used to represent a color are often called RGB values.

Because you can use any combination of 0 to 255 for each of the three primary colors, this means Pygame can draw 16,777,216 different colors (that is, 256 x 256 x 256 colors).

For example, we will create the tuple (0, 0, 0) and store it in a variable named BLACK. With no amount of red, green, or blue, the resulting color is completely black. The color black is the absence of any color. The tuple (255, 255, 255) for a maximum amount of red, green, and blue to result in white. The color white is the full combination of red, green, and blue.

The tuple (255, 0, 0) represents the maximum amount of red but no amount of green and blue, so the resulting color is red. Similarly, (0, 255, 0) is green and (0, 0, 255) is blue.

The values for each color range from 0 – 255, a total of 256 values. You can find the total number of possible color combinations by evaluating 256 x 256 x 256, which results in a value well over 16 million.

You can mix the amount of red, green, and blue to form other colors. Here are the RGB values for a few common colors:

Color	RGB Values
Aqua	(0, 255, 255)
Black	(0, 0, 0)
Blue	(0, 0, 255)
Fuchsia	(255, 0, 255)
Gray	(128, 128, 128)
Green	(0, 128, 0)
Lime	(0, 255, 0)
Maroon	(128, 0, 0)
Navy Blue	(0, 0, 128)
Olive	(128, 128, 0)
Purple	(128, 0, 128)
Red	(255, 0, 0)
Silver	(192, 192, 192)
Teal	(0, 128, 128)
White	(255, 255, 255)
Yellow	(255, 255, 0)

To use colors in Pygame, we first create Color objects using RGB values. RGB values must be in a tuple format, with three values, each corresponding to a respective color. Here are some examples below showing some color objects in pygame.

```
BLACK = pygame.Color(0, 0, 0)          # Black
WHITE = pygame.Color(255, 255, 255)    # White
GREY = pygame.Color(128, 128, 128)     # Grey
RED = pygame.Color(255, 0, 0)          # Red
```

The **fill(color)** method is used to fill in objects. For instance, assigning a rectangle the color green will only turn the borders green. If you use the **fill()** method and pass a green color object, the rectangle will become completely green.

Drawing Functions

Drawing functions are used to create objects in Pygame. Due to their similarities, they often share some parameters which they use to create the required shape.

Here are some of the common parameters.

- **surface** parameter is the surface object on which pygame will draw the shape.
- **color** parameter is the designated color of the assigned shape.
- The **pointlist** parameter is a tuple containing coordinates or "points". For instance, for a rectangle you will pass a tuple with 4 coordinate pairs inside it.
- **width** is an optional parameter that determines the size of the outline of the shape in integer values.
- **start_point** and **end_point** represent a set of coordinates. The line begins at one set of coordinates and ends at another. Likewise, **center_point** is the origin point of a circle.

```
pygame.draw.polygon(surface, color, pointlist, width)
pygame.draw.line(surface, color, start_point, end_point, width)
pygame.draw.lines(surface, color, closed, pointlist, width)
pygame.draw.circle(surface, color, center_point, radius, width)
pygame.draw.ellipse(surface, color, bounding_rectangle, width)
pygame.draw.rect(surface, color, rectangle_tuple, width)
```

Frames per Second

Computers are extremely fast and can complete millions of loop cycles in under a second. Obviously, this is a bit fast for us humans. As reference, movies are run at 24 frames per second. Anything less than that will have obvious stutter to it, and values over 100 may cause the things to move too fast for us to see.

The frame rate or refresh rate is the number of pictures that the program draws per second, and is measured in FPS or frames per second. (On computer monitors, the common name for FPS is hertz. Many monitors have a frame rate of 60 hertz, or 60 frames per second.) A low frame rate in video games can make the game look choppy or jumpy. If the program has too much code to run to draw to the screen frequently enough, then the FPS goes down.

If we do not create a limitation, the computer will execute the game loop as many times as in can within a second. To limit it we use the **tick(fps)** method where **fps** is an integer. The **tick()** method belongs to the **pygame.time.Clock** class and must be used with an object of this class.

```
FPS = pygame.time.Clock()
FPS.tick(60)
```

The frames per second can vary from game to game, depending on how it was designed. A good value to aim for is between 30 – 60. Keep in mind, that if you create a rather complex and heavy game the computer might not be able to run it well at higher frames.

A **pygame.time.Clock** object can help us make sure our program runs at a certain maximum FPS. This **Clock** object will ensure that our game programs don't run too fast by putting in small pauses on each iteration of the game loop. If we didn't have these pauses, our game program would run as fast as the computer could run it. This is often too fast for the player, and as computers get faster they would run the game faster too. A call to the `tick()` method of a `Clock` object in the game loop can make sure the game runs at the same speed no matter how fast of a computer it runs on.

```
CLOCK = pygame.time.Clock()
```

The Clock object's `tick()` method should be called at the very end of the game loop, after the call to `pygame.display.update()`. The length of the pause is calculated based on how long it has been since the previous call to `tick()`, which would have taken place at the end of the previous iteration of the game loop. (The first time the `tick()` method is called, it doesn't pause at all.)

All you need to know is that you should call the `tick()` method once per iteration through the game loop at the end of the loop. Usually this is right after the call to `pygame.display.update()`.

```
CLOCK.tick(FPS)
```

Try modifying the `FPS` constant variable to run the same program at different frame rates. Setting it to a lower value would make the program run slower. Setting it to a higher value would make the program run faster.

Pong 2: Stationary Ball

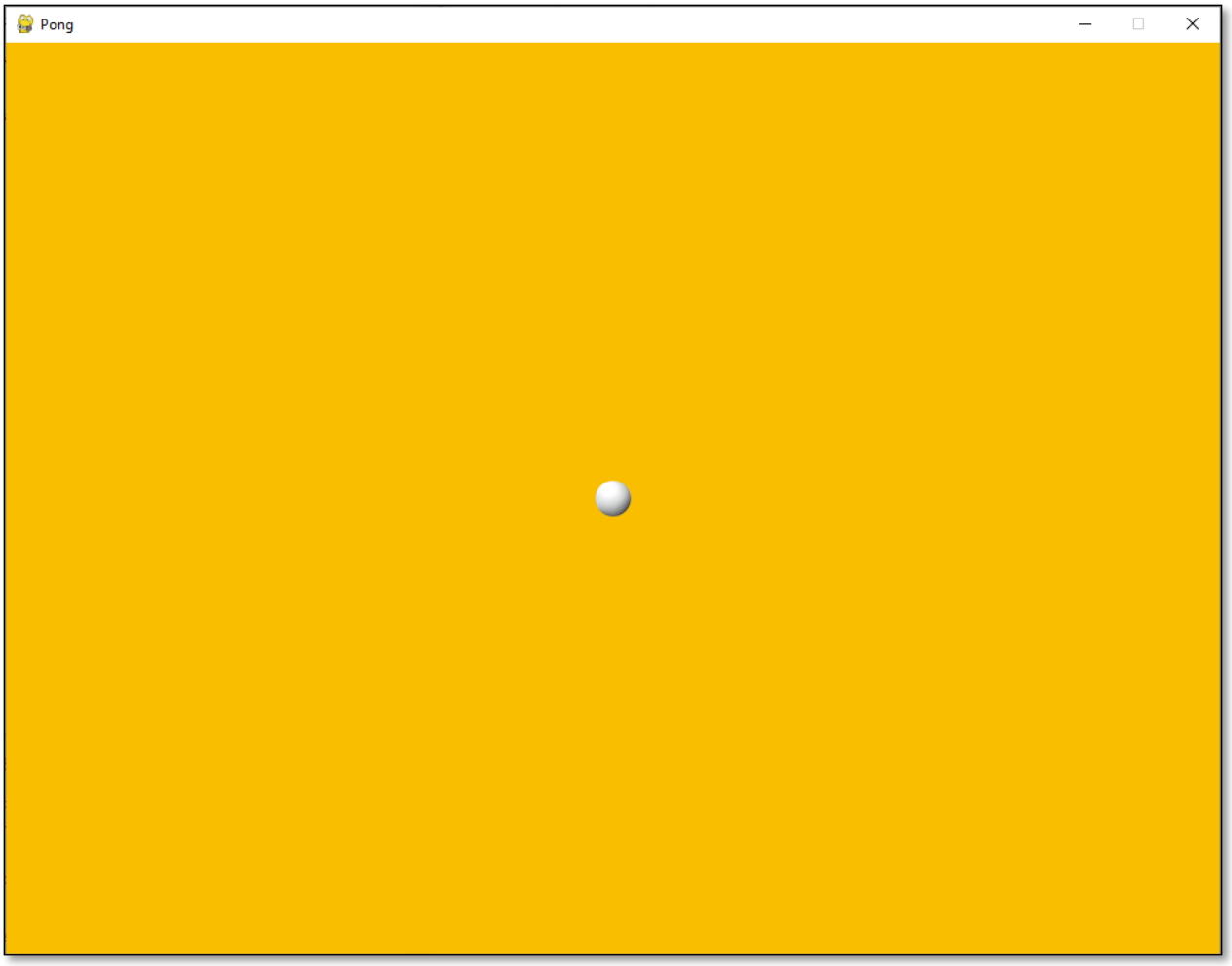
Pygame provides several different functions for drawing different shapes onto a surface object. These shapes such as rectangles, circles, ellipses, lines, or individual pixels are often called drawing primitives.

We will continue to build up on our Pong game. The **ball.png** file is attached to the assignment.

```python
"""
    Name: pong2.py
    Author:
    Date:
    Purpose: Draw a stationary Pygame Ball
"""

# Import pygame libraries
import pygame
from pygame.locals import *
# Import sys.exit to exit program
from sys import exit


class Pong:

    def __init__(self):
        """ Initialize the Pong class """
        # RGB color and screen size constants
        self.COUGAR_BLUE = (0, 58, 112)
        self.COUGAR_GOLD = (249, 190, 0)
        self.SCREEN_WIDTH = 1024
        self.SCREEN_HEIGHT = 768

        # Constant for Frames Per Second (FPS)
        self.FPS = 60

        # Initialize pygame for action
        pygame.init()

        # Setup a computer clock object
        # Keeps the game running at a constant speed
        # regardless of computer speed
        self.CLOCK = pygame.time.Clock()

        # Create display surface/Screen
        self.SCREEN = pygame.display.set_mode(
            (self.SCREEN_WIDTH,
             self.SCREEN_HEIGHT)
        )

        # Load the image into pygame
        self.ball = pygame.image.load("ball.png")
        # Get a rectangle that surrounds the image for manipulation
        self.ball_rect = self.ball.get_rect()
        # Center rect on x and y in the middle of the screen
        self.ball_rect.centerx = self.SCREEN_WIDTH/2
        self.ball_rect.centery = self.SCREEN_HEIGHT/2

        pygame.display.set_caption("Pong")
```

```
52 #----------------------------- RUN GAME ----------------------------#
53     def run_game(self):
54         """ Start the infinite Game Loop """
55         while True:
56             # Listens for program events
57             for event in pygame.event.get():
58                 # Exit game if window is closed
59                 if event.type == QUIT:
60                     # Quit Pygame
61                     pygame.quit()
62                     # Exit Python
63                     exit()
64
65             # Draw everything on the backbuffer first
66             # Fill the surface with Cougar Gold
67             self.SCREEN.fill(self.COUGAR_GOLD)
68
69             # Draw the image to the backbuffer
70             self.SCREEN.blit(
71                 self.ball,      # Source image
72                 self.ball_rect  # Destination for image
73             )
74
75             # Redraw the surface from the backbuffer
76             pygame.display.update()
77
78             # How often the game loop executes
79             self.CLOCK.tick(self.FPS)
80
81
82 #---------------------------- MAIN PROGRAM ----------------------------#
83 # Create game instance/object
84 pong = Pong()
85 # Start the game with the run_game method
86 pong.run_game()
```

When this program is run, the following window is displayed until the user closes the window:



You did it again! You created the world's second most boring game. You turned the screen Cougar Gold and drew a Cougar Blue ball.

We created constants for each of the colors. Doing this makes our code more readable. Seeing **COUGAR_GOLD** in the source code is much easier to understand than **(0, 58, 112)**.

Rect Object

Instead of drawing the ball directly to the screen, we create a Rect object that we draw the ball in.

The code below creates a Rect object with a top left corner at (10, 20) that is 200 pixels wide and 300 pixels tall:

```
spamRect = pygame.Rect(10, 20, 200, 300)
spamRect == (10, 20, 200, 300)
```

The handy thing about this is that the Rect object automatically calculates the coordinates for other features of the rectangle. Rect and other objects are easier to find and move around the game screen.

If you need to know the X coordinate of the right edge of the pygame.Rect object we stored in the spamRect variable, you access the Rect object's right attribute:

```
>>> spamRect.right
210
```

The Pygame code for the Rect object automatically calculated that if the left edge is at the X coordinate 10 and the rectangle is 200 pixels wide, then the right edge must be at the X coordinate 210. If you reassign the right attribute, all the other attributes are automatically recalculated. If we used drawing objects, we would have to calculate this manually.

```
>>> spam.right = 350
>>> spam.left
150
```

Here's a list of all the attributes that pygame.Rect objects provide:

Attribute Name	**Description**
myRect.left	The int value of the X-coordinate of the left side of the rectangle.
myRect.right	The int value of the X-coordinate of the right side of the rectangle.
myRect.top	The int value of the Y-coordinate of the top side of the rectangle.
myRect.bottom	The int value of the Y-coordinate of the bottom side.
myRect.centerx	The int value of the X-coordinate of the center of the rectangle.
myRect.centery	The int value of the Y-coordinate of the center of the rectangle.
myRect.width	The int value of the width of the rectangle.
myRect.height	The int value of the height of the rectangle.

myRect.size	A tuple of two ints: (width, height)
myRect.topleft	A tuple of two ints: (left, top)
myRect.topright	A tuple of two ints: (right, top)
myRect.bottomleft	A tuple of two ints: (left, bottom)
myRect.bottomright	A tuple of two ints: (right, bottom)
myRect.midleft	A tuple of two ints: (left, centery)
myRect.midright	A tuple of two ints: (right, centery)
myRect.midtop	A tuple of two ints: (centerx, top)
myRect.midbottom	A tuple of two ints: (centerx, bottom)

Primitive Drawing Functions

The drawing functions are named after the shapes they draw. The parameters you pass these functions tell them which Surface object to draw on, where to draw the shape (and what size), in what color, and how wide to make the lines.

We used the **pygame.draw.circle** function for our stationary ball program

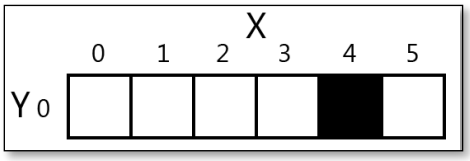
- **pygame.draw.circle(surface, color, center_point, radius, width)** – This function draws a circle. The center of the circle is at the **center_point** parameter. The integer passed for the radius parameter sets the size of the circle.
- The radius of a circle is the distance from the center to the edge. (The radius of a circle is always half of the diameter.) Passing 20 for the radius parameter will draw a circle that has a radius of 20 pixels.

Animation

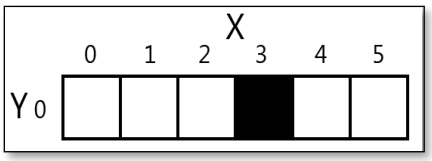
Now that we know how to get the Pygame framework to draw to the screen, let's learn how to make animated pictures. A game with only still, unmoving images is dull. (Sales of the game "Look At This Rock" have been disappointing.)

Animated images are the result of drawing an image on the screen, then a split second later drawing a slightly different image on the screen. Imagine the program's window was 6

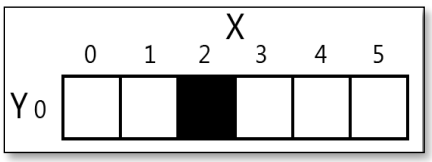
pixels wide and 1 pixel tall, with all the pixels white except for a black pixel at 4, 0. It would look like this:



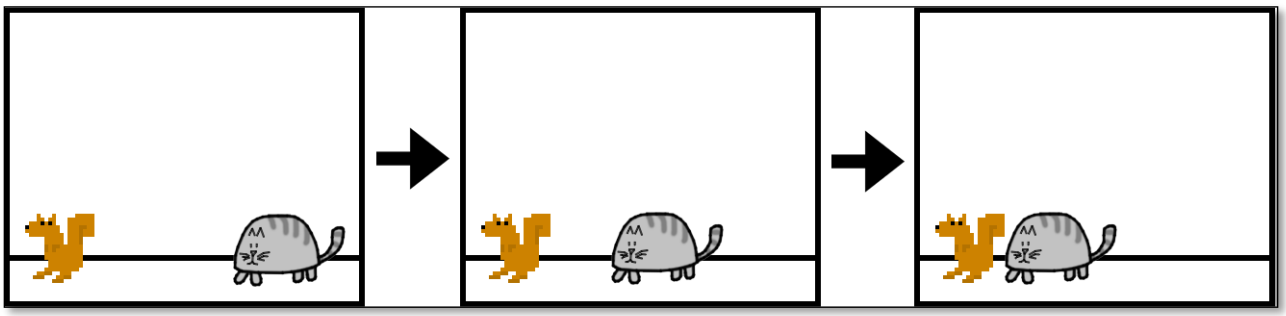
If you changed the image so that 3, 0 was black and 4, 0 was white, it would look like this:



To the user, it looks like the black pixel has "moved" over to the left. If you redrew the window to have the black pixel at 2, 0, it would continue to look like the black pixel is moving left:



It may look like the black pixel is moving, but this is just an illusion. To the computer, it is just showing three different images that each just happen to have one black pixel. Consider if the three following images were rapidly shown on the screen:



To the user, it would look like the cat is moving towards the squirrel. But to the computer, they're just a bunch of pixels. The trick to making believable looking animation is to have your program draw a picture to the window, wait a fraction of a second, and then draw a slightly different picture.

Pong 3: Moving Ball

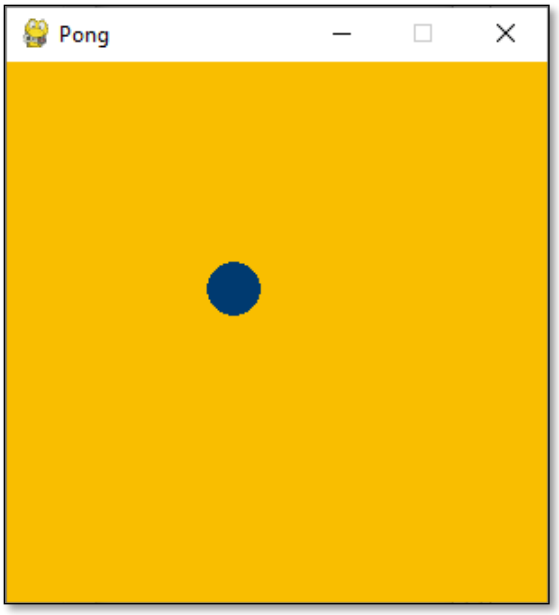
Yes, we are finally going to make something move. By changing the (x, y) values each time through the game loop, we animate our ball.

Open **pong2.py** Save as **pong3.py** Change the code to the following:

```
"""
    Name: pong3.py
    Author:
    Date:
    Purpose: Move a Pygame Ball across the screen
"""

# Import pygame libraries
import pygame
from pygame.locals import *
# Import sys.exit to exit program
from sys import exit


class Pong:

    def __init__(self):
        """ Initialize the Pong class """
        # RGB color and screen size constants
        self.COUGAR_BLUE = (0, 58, 112)
        self.COUGAR_GOLD = (249, 190, 0)
        self.SCREEN_WIDTH = 1024
        self.SCREEN_HEIGHT = 768

        # Constant for Frames Per Second (FPS)
        self.FPS = 60

        # Initialize pygame
        pygame.init()

        # Setup a computer clock object
        # Keeps the game running at a constant speed
        # regardless of computer speed
        self.CLOCK = pygame.time.Clock()

        # Create display surface/Screen
        self.SCREEN = pygame.display.set_mode(
            (self.SCREEN_WIDTH,
             self.SCREEN_HEIGHT)
        )

        # Load the image into pygame
        self.ball = pygame.image.load("ball.png")
        # Get a rectangle that surrounds the image for manipulation
        self.ball_rect = self.ball.get_rect()
        # Center rect on x and y in the middle of the screen
        self.ball_rect.centerx = self.SCREEN_WIDTH/2
        self.ball_rect.centery = self.SCREEN_HEIGHT/2
        pygame.display.set_caption("Pong")

```

```python
#---------------------------- RUN GAME ----------------------------#
    def run_game(self):
        """ Start the infinite Game Loop """
        while True:
            # Listens for program events
            for event in pygame.event.get():
                # Exit game if window is closed
                if event.type == QUIT:
                    pygame.quit()
                    exit()

            # Draw everything on the backbuffer first
            # Fill the display surface with Cougar Gold
            self.SCREEN.fill(self.COUGAR_GOLD)

            # Move (x, y) coordinates of the ball 2 pixels each loop
            self.ball_rect.x = self.ball_rect.x + 2
            self.ball_rect.y = self.ball_rect.y + 2

            # Draw the image to the backbuffer
            self.SCREEN.blit(
                self.ball,      # Source image
                self.ball_rect  # Destination location of image
            )

            # Redraw the surface from the backbuffer
            pygame.display.update()

            # How often the game loop executes
            self.CLOCK.tick(self.FPS)


#---------------------------- MAIN PROGRAM ----------------------------#
# Create game instance/object
pong = Pong()
# Start the game with the run_game method
pong.run_game()
```

Example run:



Yes, we have liftoff! You can't tell by the screenshot. Our ball is moving down and to the right 1 pixel a frame. Our Frames per Second is 60; the ball is moving 60 pixels a second.

Notice that we are using **surface.fill(COUGAR_GOLD)** to clear the screen each time through the game loop. Comment that line out to see why.

Pong 4: Moving Ball Class

This version of the Moving Ball program will not look any different than the last one. We will modify the single file to incorporate OOP (Object Oriented Programming) concepts with separate class files.

Two advantages of using OOP is to separate the game components into separate files and create multiple "sprites" from a single class. We can create hundreds of aliens from a single Alien class.

What is a sprite? A **sprite** is a two-dimensional image that is part of the larger graphical scene. Typically, a **sprite** will be an object of some sort **in** the scene that will be interacted with like a car, frog, or little plumber guy.

Create a file called **ball.py** with the following code. Some can be copied from **pong.py**

```python
"""
    Name: ball.py
    Author:
    Date:
    Purpose: All logic for the ball is in this class
"""

# Import modules
import pygame
from pygame.locals import *


class Ball(pygame.sprite.Sprite):
    """ Define the Ball class, methods and fields """

    def __init__(self, pong):
        """
            Initialize a Ball object
            Include pong reference
        """
        # Create reference to pong.config object
        self.config = pong.config
        self.SCREEN = pong.SCREEN

        # Load the image into pygame
        self.ball = pygame.image.load("ball.png")
        # Get a rectangle that surrounds the image for manipulation
        self.ball_rect = self.ball.get_rect()
        # Center rect on x and y in the middle of the screen
        self.ball_rect.centerx = self.config.SCREEN_WIDTH/2
        self.ball_rect.centery = self.config.SCREEN_HEIGHT/2

        # Initialize a Ball object from Sprite class
        super().__init__()

#---------------------------- DRAW BALL ----------------------------#
    def draw_ball(self, surface):
        """ Draw the Ball object on the surface """
        # Move x, y coordinates at speed pixel each loop
        self.ball_rect.x = self.ball_rect.x + self.config.speed
        self.ball_rect.y = self.ball_rect.y + self.config.speed

        # Draw the image to the backbuffer
        self.SCREEN.blit(
            self.ball,      # Source image
            self.ball_rect  # Destination image location
        )
```

Notice how all the game logic for the Ball is now in the Ball class.

Create a file named **config.py** This file contains shared configurations for all classes.

```python
"""
    Filename: config.py
    Author:
    Date:
    Purpose: Object variables and constants for the entire program
"""


class Config:

    def __init__(self):
        """ Setup the object data fields """
        # Assign FPS (Frames per Second) a value
        self.FPS = 60

        # Pixels of movement
        self.speed = 2

        # Set the size of the surface/screen
        self.SCREEN_WIDTH = 1024
        self.SCREEN_HEIGHT = 768

        # Screen size as a list
        self.SCREEN_SIZE = [
            self.SCREEN_WIDTH,
            self.SCREEN_HEIGHT
        ]

        # Setup color constants
        self.COUGAR_GOLD = (249, 190, 0)
        self.COUGAR_BLUE = (0, 58, 112)
```

Open **pong3.py** Save as **pong4.py** Change the code to the following:

```
"""
    Name: pong4.py
    Author:
    Date:
    Purpose: Move an onscreen sprite with ball class
    Refactored pong into ball and config class
"""

# Import pygame libraries
import pygame
from pygame.locals import *
# Import sys.exit to exit program
from sys import exit
# import ball and config modules
from ball import Ball
from config import Config


class Pong:

    def __init__(self):
        """ Initialize the Pong class"""
        pygame.init()
        # Initialize Config object
        # for access to data fields and methods
        self.config = Config()

        # Create clock object
        self.CLOCK = pygame.time.Clock()

        # Setup a display surface with caption
        self.SCREEN = pygame.display.set_mode(
            self.config.SCREEN_SIZE
        )
        # Initialize Ball object
        self.ball = Ball(self)
        pygame.display.set_caption("Pong")

#---------------------------- RUN GAME ----------------------------#
    def run_game(self):
        """ Game loop """
        while True:
            # Check events and update screen
            self.__check_events()
            self.__update_screen()
```

```python
#------------------------- CHECK EVENTS ----------------------------#
    def __check_events(self):
        """ Listen for program events """
        for event in pygame.event.get():

            # Exit game if window is closed
            if event.type == QUIT:
                pygame.quit()
                exit()

#------------------------- UPDATE SCREEN ----------------------------#
    def __update_screen(self):
        """ Update the screen """
        # Fill the surface to clear the screen
        self.SCREEN.fill(self.config.COUGAR_GOLD)

        # Use the ball object method to draw on the surface
        self.ball.draw_ball(self.SCREEN)

        # Update/redraw the surface
        pygame.display.update()

        # How often our game loop executes
        self.CLOCK.tick(self.config.FPS)


#---------------------------- MAIN PROGRAM ----------------------------
# Create game instance/object
pong = Pong()
# Start the game with the run_game method
pong.run_game()
```

Run the program to confirm that it works.

Pong 5: Collisions

Our ball moves around nicely. Let's have it bounce off the walls.

Assignment Submission

1. Attach the pseudocode.
2. Attach the program files.
3. Attach screenshots showing the successful operation of the program.
4. Submit in Blackboard.